

Assignment 01 — Persona-Based AI Chatbot

Prompt Engineering | Scaler Academy

Overview

You will build a **working persona-based AI chatbot** that lets users have real conversations with three Scaler/InterviewBit personalities — **Anshuman Singh**, **Kshitij Mishra**, and **Abhimanyu Saxena**. Every concept taught in the lecture must be applied inside this single application.

This is not a theoretical exercise. You will write real system prompts, call a real LLM API, and ship a working product.

The Three Personas

You must build and maintain **three separate system prompts**, one for each persona. Do your research — watch their talks, read their LinkedIn posts, Whatsapp Messages & Class Tutorials. The richer your research, the more authentic your prompt.

Persona 1 — Anshuman Singh

Persona 2 — Abhimanyu Saxena

Persona 3 — Kshitij Mishra

System Prompt Requirements

Each system prompt **must** include all of the following:

1. **Persona description** — Who is this person? Their background, values, and communication style in detail.
 2. **Few-shot examples** — Minimum 3 example user questions with ideal persona-style answers embedded directly in the system prompt.
 3. **Chain-of-Thought instruction** — Tell the model to reason step-by-step internally before delivering its answer.
 4. **Output instruction** — Specify the format and length of the response (e.g., 4–5 sentences, end with a question).
 5. **Constraints** — What the persona should never do
-

Technical Requirements

Frontend

- A clean chat interface with a **persona switcher** (tabs or buttons) for all three personalities
- When the persona is switched, the system prompt changes and the conversation resets
- The active persona must be clearly visible in the UI at all times
- Suggestion chips (quick-start questions) per persona
- Typing indicator while the API call is in progress
- Responsive — must work on mobile and desktop

Backend / API

- API key must be stored in an **environment variable** — never hardcoded in source code
- Each persona must have its own system prompt passed correctly in the API call
- Handle API errors gracefully — show a user-friendly message if the call fails

Tech Stack

You are free to use any stack you're comfortable with:

Marking Scheme — Total: 10 Points

#	Criterion	Marks	What We Look For
1	GitHub Repository	2	Clean repo structure, <code>.env.example</code> present, no API keys committed, clear README with setup steps and screenshots
2	Live Project (Deployed)	2	The app is live and accessible via a public URL (Vercel, Netlify, Railway, or similar). Persona switching works in production.
3	Frontend Quality	2	The UI is clean, intuitive, and functional. A personal switcher works. Suggestion chips present. Typing indicator present. Mobile-friendly.
4	Backend & Prompt Quality	2	API is correctly wired. Each persona has a distinct, well-researched system prompt with few-shot examples, CoT instruction, and constraints. The bot genuinely sounds like the real person.
5	Documentation & Reflection	2	<code>prompts.md</code> file in the repo with all three system prompts annotated. 300–500 word reflection covering: what worked, what the GIGO principle taught you, and what you would improve.

Submission Checklist

Before submitting, verify every item below:

- ☐ GitHub repo is **public** and link is shared
- ☐ Repo contains a `README.md` with setup instructions and a deployed link
- ☐ Repo contains a `prompts.md` with all three system prompts + inline comments
- ☐ Repo contains a `reflection.md` (300–500 words)
- ☐ `.env.example` file is present; no real API key is committed anywhere
- ☐ App is **deployed and live** — not just running locally
- ☐ All three personas are working in the live app
- ☐ Persona switching resets the conversation correctly
- ☐ App handles API errors without crashing
- ☐ UI is mobile-responsive

What Good Looks Like

A high-scoring submission will:

- Have system prompts so well-crafted that a person familiar with Anshuman, Kshitij, or Abhimanyu
- Show clear evidence of research — references to real things the person has said, built, or believes
- Have a clean, professional UI that feels like something you'd actually want to use
- Have a live deployed link that works without setup on the evaluator's machine
- Include a `prompts.md` that reads like a product decision document — explaining *why* each prompt choice was made, not just *what* was written

A low-scoring submission will:

- Have generic system prompts like "You are Anshuman Singh, be helpful and friendly" — this is a GIGO failure
 - Have API keys hardcoded or committed to the repo
 - Have no deployed link — only a local demo
 - Have no documentation or a `README.md` that just says "Run npm start"
-

Important Notes

- **Research is part of the assignment.** You are expected to spend time understanding each persona before writing a single line of their system prompt.
 - **GIGO applies to you too.** If you put lazy input into your system prompt, you will get lazy output from the LLM. There are no shortcuts.
 - **The personas are real people.** Represent them fairly and professionally. Do not put words in their mouths that contradict their known public positions.
 - **Working beats pretty.** A functional app with great prompts and a basic UI scores higher than a beautiful app with a broken or shallow chatbot.
-

Deadline & Submission

Submit your **GitHub repo link** and **live project URL** on the google form.

Any submission without a live deployed link will be considered incomplete.